

Subscribing to the Speed Control Topic

1. Experiment Objective

In this experiment, you will learn how to subscribe to a speed control topic (`cmd_vel`) of type `geometry_msgs/msg/Twist` through micro-ROS and drive the motors of the MicroROS-V2 control board to implement motion control of the robot car. Through this example, you will master the receiving and parsing of the ROS2 standard motion control message, the integration of motor drivers, and the design method of a **timeout automatic stop** safety mechanism. This example subscribes to the `/cmd_vel` topic and controls the car's forward, backward, turning and in-place rotation based on linear velocity and angular velocity.

2. Experiment Principle

1. Keyword Explanation

Concept	Description
Twist message	The ROS2 standard motion control message type, defined in <code>geometry_msgs/msg/Twist</code> , containing two 3D vector fields: linear (linear velocity) and angular (angular velocity). It is the standard input interface for robot chassis control
Differential drive	A kinematics model that achieves steering through the speed difference of the left and right independently driven wheels. The target linear velocity and angular velocity are decomposed into left and right wheel speeds using the formulas <code>speed_L = vx - vz * d/2</code> and <code>speed_R = vx + vz * d/2</code>
Timeout safety mechanism	When the micro-ROS node has not received a new <code>cmd_vel</code> message within a specified time (1500 ms in this example), it automatically sets the motor speed to 0 and stops motion, a safety measure that prevents the robot from running out of control due to communication interruption
micro-ROS	A ROS2 client library for microcontrollers, enabling resource-constrained devices such as the ESP32 to participate in the ROS2 ecosystem. It communicates with the ROS2 Agent over Wi-Fi/UDP, implementing functions such as topic publishing, subscription and service calls
PID control	A proportional-integral-derivative control algorithm, used in motor driving to precisely control wheel speed so that the actual speed approaches the target speed. This example uses the default PID parameters (Kp=1.15, Ki=0.24, Kd=0.0)
FreeRTOS Queue	The inter-task communication mechanism provided by ESP32 FreeRTOS. This example uses a queue of length 1 to pass the latest speed command between the micro-ROS task and the motion control task, ensuring the latest control command is always executed

2. Technical Architecture

This example adopts the **ESP32** → **micro-ROS** → **Agent** → **ROS2 host** communication chain. The control board connects via Wi-Fi to the ROS2 host running the micro-ROS Agent and subscribes to the `/cmd_vel` topic to obtain speed control commands.

Configuration Item	Value	Description
Node name	<code>twist_subscriber</code>	ROS2 node name
Topic name	<code>cmd_vel</code>	The subscribed ROS2 standard speed control topic
Message type	<code>geometry_msgs/msg/Twist</code>	Contains linear.x (linear velocity) and angular.z (angular velocity)
QoS mode	Reliable (default)	Control-type messages require reliable transmission
Timeout stop	1500 ms	Automatically stops all motors after a timeout
Queue length	1	Always executes the latest speed command

In terms of software architecture, the program adopts a **multi-task separation + timeout protection** design:

- `micro_ros_task` is responsible for ROS2 communication (subscribing to `cmd_vel`)
- `motion_worker_task` is responsible for motor control and timeout detection (20 ms period)
- The two are decoupled through a FreeRTOS queue and do not block each other

3. Core Topic Quick Reference

Firmware Downlink (Control Commands)

Topic	Message Type	QoS	Field	Range	Direction
<code>/cmd_vel</code>	<code>geometry_msgs/msg/Twist</code>	Reliable	<code>linear.x</code> (linear velocity m/s)	[-1.0, 1.0]	<code>+=</code> forward, <code>-=</code> backward
			<code>angular.z</code> (angular velocity rad/s)	[-6.0, 6.0]	<code>+=</code> turn left, <code>-=</code> turn right

Timeout safety: If no new `cmd_vel` is received within 1500 ms, the car stops automatically.

Usage Example:

```
# The car moves forward at 0.3 m/s
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3}, angular: {z: 0.0}}"
# The car stops
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"
```

4. Middleware Configuration

micro-ROS uses a **static memory model** (`RCUTILS_AVOID_DYNAMIC_ALLOCATION=ON`), all entity memory is pre-allocated at compile time, and runtime dynamic creation is not supported.

Configuration file: `~/esp/extra_components/micro_ros_espidf_component/colcon.meta`

`RMW_UXRCE_TRANSPORT`	`udp`	UDP transport (Wi-Fi communication)
`RMW_UXRCE_MAX_PUBLISHERS`	`6`	Maximum number of publishers
`RMW_UXRCE_MAX_SUBSCRIPTIONS`	`6`	Maximum number of subscribers
`RMW_UXRCE_MAX_NODES`	`1`	Maximum number of nodes
`RMW_UXRCE_MAX_SERVICES`	`0`	Services not supported
`RMW_UXRCE_MAX_CLIENTS`	`0`	Clients not supported
`RMW_UXRCE_MAX_HISTORY`	`1`	History depth = 1, no message queuing
`UCLIENT_UDP_TRANSPORT_MTU`	`4096`	Maximum UDP packet size 4 KB
`UCLIENT_MIN_HEARTBEAT_TIME_INTERVAL`	`1`	Agent heartbeat interval 1 ms

Key Constraints:

- `MAX_HISTORY=1` means there is no message buffer queue, so uplink topics (firmware → host) uniformly use **best_effort** QoS
- `MAX_SERVICES=0`, `MAX_CLIENTS=0` means only topic communication is supported, not Service/Client
- After modifying `colcon.meta`, the micro-ROS library must be recompiled for the changes to take effect: run `idf.py clean-microros` in the project directory to clean and rebuild the micro-ROS library, then `idf.py build` to compile the firmware

3. Experiment Steps

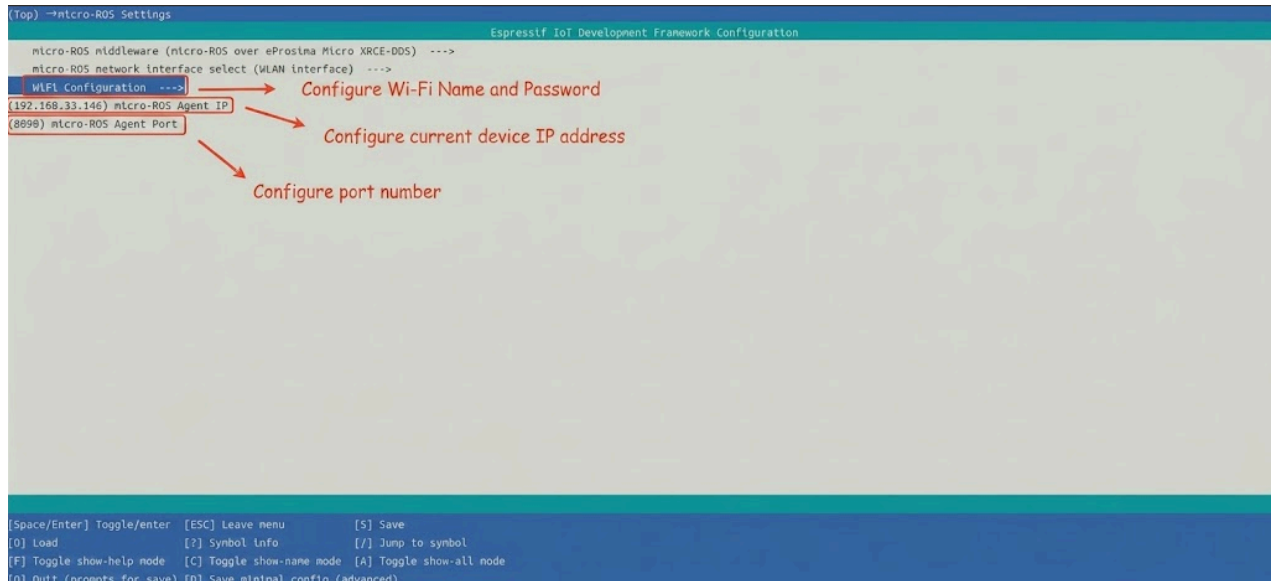
1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✔ Required
USB Type-C cable	✔ Required
Wi-Fi network (2.4 GHz)	✔ Required
DC geared motor ×2	✔ Required
Motor driver board	✔ Required

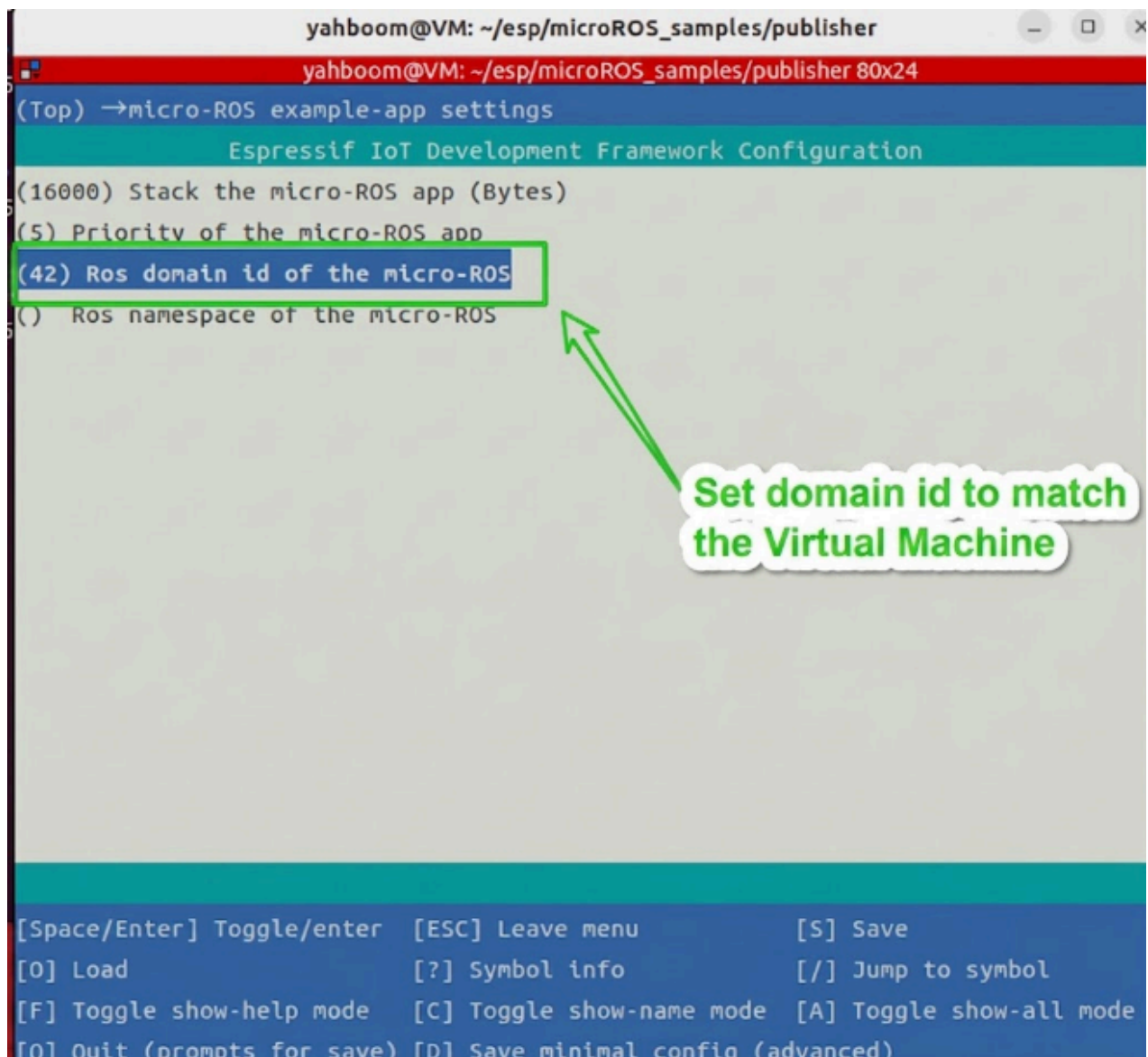
2. Prerequisites

1. The ESP32-IDF development environment has been set up
2. The MicroROS-V2 control board is connected to the computer via USB, and the motors are correctly connected
3. Configure the Wi-Fi SSID, password, Agent IP and port via `idf.py menuconfig` (the `micro_ros` menu item)

```
cd ~/esp/microROS_samples/twist_subscriber
source ~/esp/esp-idf/export.sh
idf.py menuconfig
```



4. Select example-app setting to set the domain ID to be consistent with the virtual machine



After configuration, press "s" to save and press "q" to exit the configuration interface.

3. Start micro_ros_agent (Terminal 1)

Start the micro-ROS Agent on the ROS2 host:

```
cd ~/uros_ws
source install/local_setup.bash
ros2 run micro_ros_agent micro_ros_agent udp4 --port 8090 -v4
```

4. Build and Flash (Terminal 2)

```
cd ~/esp/microROS_samples/twist_subscriber
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

5. How to Stop

1. First send a stop command to ensure the car is stationary: `ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}" -1`
2. Press `Ctrl+J` in Terminal 2 to exit the serial monitor (stop firmware execution)
3. Press `Ctrl+C` in Terminal 1 to stop micro_ros_agent

Safety tip: Timeout automatic stop (1500 ms) is a backup mechanism; proactively sending a stop command is the safest way.

4. Experiment Result

After the build and flash succeed, publish speed control commands on the ROS2 host:

```
# The car moves forward at 0.3 m/s
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3}, angular: {z: 0.0}}"

# The car rotates in place to the left at 0.5 rad/s
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.5}}"

# The car stops
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}"
```

Motion mode comparison table:

linear.x	angular.z	Motion effect
> 0	0	Forward
< 0	0	Backward
0	> 0	Rotate left in place

linear.x	angular.z	Motion effect
0	< 0	Rotate right in place
> 0	> 0	Forward + turn left (arc motion)
> 0	< 0	Forward + turn right (arc motion)

The control board serial port will output:

```
I (xxx) MAIN: Agent reachable: 192.168.x.x:8888
I (xxx) MAIN: micro-ROS cmd_vel subscriber init done
I (xxx) MAIN: cmd_vel: vx=0.300, wz=0.000
```

When no command is sent for more than 1.5 seconds:

```
W (xxx) MAIN: cmd_vel timeout: 1500 ms
W (xxx) MAIN: cmd_vel timeout , stop motion
```

5. Program Analysis

Source code paths:

- Firmware source code: `~/esp/microROS_samples/twist_subscriber/main/main.c`
- Motion control:
`~/esp/microROS_samples/twist_subscriber/components/car_motion/car_motion.h`,
`car_motion.c`
- Motor driver: `~/esp/microROS_samples/twist_subscriber/components/motor/motor.h`,
`motor.c`

1. Main Program Logic

`main/main.c` adopts a **multi-task separation + timeout protection** design:

1. In `app_main()`:

- Calls `Motor_Init()` to initialize the motor driver
- Creates the FreeRTOS queue `s_motion_queue` (length 1, always keeps the latest command)
- Creates the `motion_worker_task` task, which is responsible for motor control and timeout detection
- Initializes the Wi-Fi network
- Creates the `micro_ros_task` task

2. In `micro_ros_task`:

- Creates the node `twist_subscriber`
- Subscribes to the topic `cmd_vel` in **Reliable mode**, with message type `geometry_msgs/msg/Twist`
- The subscription callback `twist_callback` forwards the speed command to the execution task

through the queue

3. In `motion_worker_task`:

- Checks for a new command every 20 ms
- Calls `Motion_Ctrl()` to execute motion control
- Continuously monitors command freshness: if no new command is received within 1500 ms, it stops the car automatically

2. Key Components

`components/car_motion/car_motion.h` — Motion control interface:

Parameter definition file:

`~/esp/microROS_samples/twist_subscriber/components/car_motion/car_motion.h`

Parameter	Value	Description
<code>ROBOT_WIDTH</code>	0.1714 m	Distance between the centers of the left and right wheels
<code>ROBOT_HALF_WIDTH</code>	0.0857 m	Half width, used for differential calculation
<code>ROBOT_MAX_LINEAR_SPEED</code>	1.0 m/s	Maximum linear velocity
<code>ROBOT_MAX_ANGULAR_SPEED</code>	6.0 rad/s	Maximum angular velocity
<code>ROBOT_TURN_RADIUS</code>	0.30 m	Turning radius
<code>ROBOT_SPIN_MAX_WZ</code>	3.0 rad/s	Maximum in-place rotation angular velocity

`components/car_motion/car_motion.c` — Core motion control functions:

Function	Description
<code>Motion_Ctrl(float v_x, float v_y, float v_z)</code>	Controls car motion: Vx is linear velocity [-1.0, 1.0] m/s, Vy is invalid, Vz is angular velocity [-5.0, 5.0] rad/s
<code>Motion_Stop(uint8_t brake)</code>	Stops the car (optionally coasting or braking)
<code>Motion_Ctrl_State(uint8_t state, float speed)</code>	Controls by motion state (forward/backward/turn left/turn right/rotate in place/brake)
<code>Motion_Get_Speed(car_motion_t* car)</code>	Gets the current actual speed

Differential kinematics model `Motion_Ctrl`:

Source file: `~/esp/microROS_samples/twist_subscriber/components/car_motion/car_motion.c`

```

void Motion_Ctrl(float V_x, float V_y, float V_z)
{
    line_v = limit_linear_speed(V_x);
    angular_v = limit_angular_speed(V_z);

    // Differential decomposition: left wheel speed = vx - vz × half width
    speed_L_setup = line_v - angular_v * ROBOT_HALF_WIDTHH;
    // Right wheel speed = vx + vz × half width
    speed_R_setup = line_v + angular_v * ROBOT_HALF_WIDTHH;

    Motor_Set_Speed(speed_L_setup, speed_R_setup);
}

```

The timeout automatic stop mechanism of `motion_worker_task`:

Source file: `~/esp/microROS_samples/twist_subscriber/main/main.c`

```

#define CMD_VEL_TIMEOUT_MS          1500

static void motion_worker_task(void *arg)
{
    (void)arg;

    motion_cmd_t cmd = {0};
    int64_t last_cmd_us = 0;
    bool stopped_by_timeout = false;

    while (1) {
        /* Check every 20 ms whether there is a new command */
        if (xQueueReceive(s_motion_queue, &cmd, pdMS_TO_TICKS(20)) == pdTRUE) {
            last_cmd_us = cmd.stamp_us;
            stopped_by_timeout = false;

            Motion_Ctrl(cmd.linear_x, 0.0f, cmd.angular_z);

            ESP_LOGI(TAG, "cmd_vel: vx=%.3f, wz=%.3f",
                      (double)cmd.linear_x, (double)cmd.angular_z);
        }

        /* Timeout automatic stop to prevent the car from continuing to move after the host
        disconnects */
        if (!stopped_by_timeout && last_cmd_us > 0) {
            int64_t now_us = esp_timer_get_time();
            if ((now_us - last_cmd_us) > ((int64_t)CMD_VEL_TIMEOUT_MS * 1000)) {
                int64_t timeout_ms = (now_us - last_cmd_us) / 1000;
                ESP_LOGW(TAG, "cmd_vel timeout: %lld ms", timeout_ms);
                Motion_Ctrl(0.0f, 0.0f, 0.0f);
                stopped_by_timeout = true;
                ESP_LOGW(TAG, "cmd_vel timeout , stop motion");
            }
        }
    }
}

```



```
}
```

`twist_callback` **subscription callback:**

Source file: `~/esp/microROS_samples/twist_subscriber/main/main.c`

```
static void twist_callback(const void *msgin)
{
    if (msgin == NULL || s_motion_queue == NULL) {
        return;
    }

    const geometry_msgs__msg__Twist *msg =
        (const geometry_msgs__msg__Twist *)msgin;

    motion_cmd_t cmd = {
        .linear_x = (float)msg->linear.x,
        .angular_z = (float)msg->angular.z,
        .stamp_us = esp_timer_get_time(),
    };

    /* Queue length is 1, always overwrites with the latest command */
    (void)xQueueOverwrite(s_motion_queue, &cmd);
}
```

`components/motor/motor.h` — Key parameters of the motor driver interface:

Parameter definition file: `~/esp/microROS_samples/twist_subscriber/components/motor/motor.h`

Parameter	Value	Description
<code>WHEEL_DIAMETER</code>	67.5 mm	Wheel diameter
<code>MOTOR_ENCODER_CIRCLE</code>	1320	Encoder pulses per revolution
<code>MOTOR_PID_PERIOD</code>	10 ms	PID control period
<code>MOTOR_MAX_SPEED</code>	1.0 m/s	Maximum speed
<code>MOTOR_PID_DEFAULT_KP</code>	1.15	Default proportional gain
<code>MOTOR_PID_DEFAULT_KI</code>	0.24	Default integral gain
<code>MOTOR_PID_DEFAULT_KD</code>	0.0	Default derivative gain

6. Common Problems

Problem	Cause	Solution
The car does not move	The motor driver is not initialized or the power supply is insufficient	Check the motor power connections and confirm the battery has sufficient charge

Problem	Cause	Solution
Linear or angular velocity is limited	Exceeding the maximum limit is clamped	Confirm the linear velocity does not exceed 1.0 m/s and the angular velocity does not exceed 6.0 rad/s
The car stops automatically after commands stop being sent	The timeout automatic stop mechanism is triggered	This is a safety protection mechanism; <code>cmd_vel</code> must be sent continuously at a frequency of at least 1 Hz
The actual speed of the car does not match the command	PID parameters are not tuned	Adjust the PID parameters with <code>Motor_Update_PID_Parm()</code>
Incorrect motion direction	The motor wiring sequence is reversed	Adjust the motor wiring order or modify the <code>MOTOR_ENCODER_SIGN_M1/M2</code> signs

Safety tip: The timeout automatic stop mechanism (1500 ms) ensures that the car can stop automatically when the host disconnects or the program malfunctions. This is a **necessary safety design** for mobile robot control. Please stay alert during testing and be ready to cut off power manually at any time.